Embedded Control for Robotics

Lecture # 03 Fundaments of Digital Controls

Chapter # 4, Page 63, Embedded Digital Control With Microcontrollers

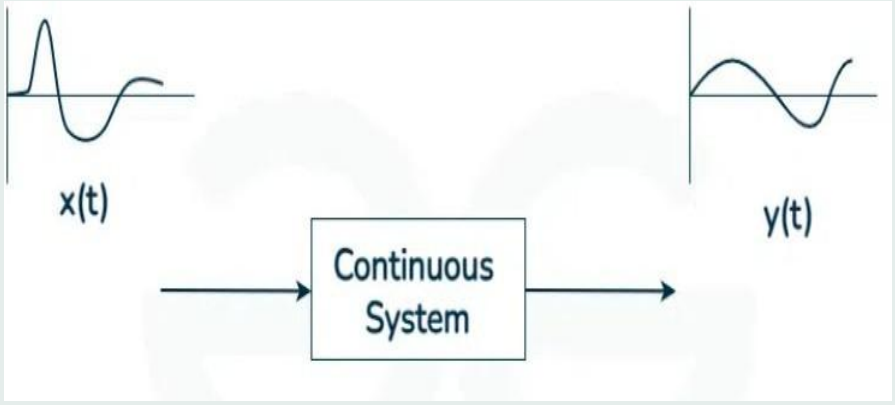
Dr. Suleman Khan
CS, NUCES-FAST, Islamabad

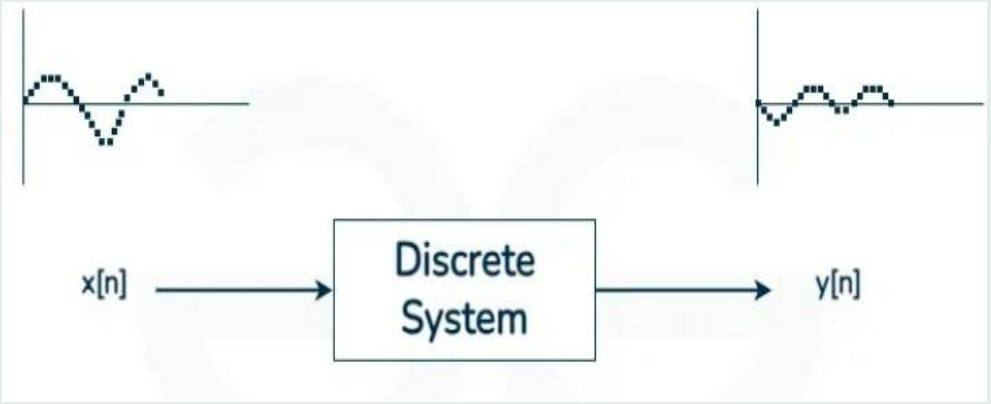
Overview- Fundamentals of Digital Control

- Digital Signals
- Types of Digital Signals
- Digital Systems
- Stability of Digital Systems
- Connections types of Digital Systems

Control Systems: Simplified Picture

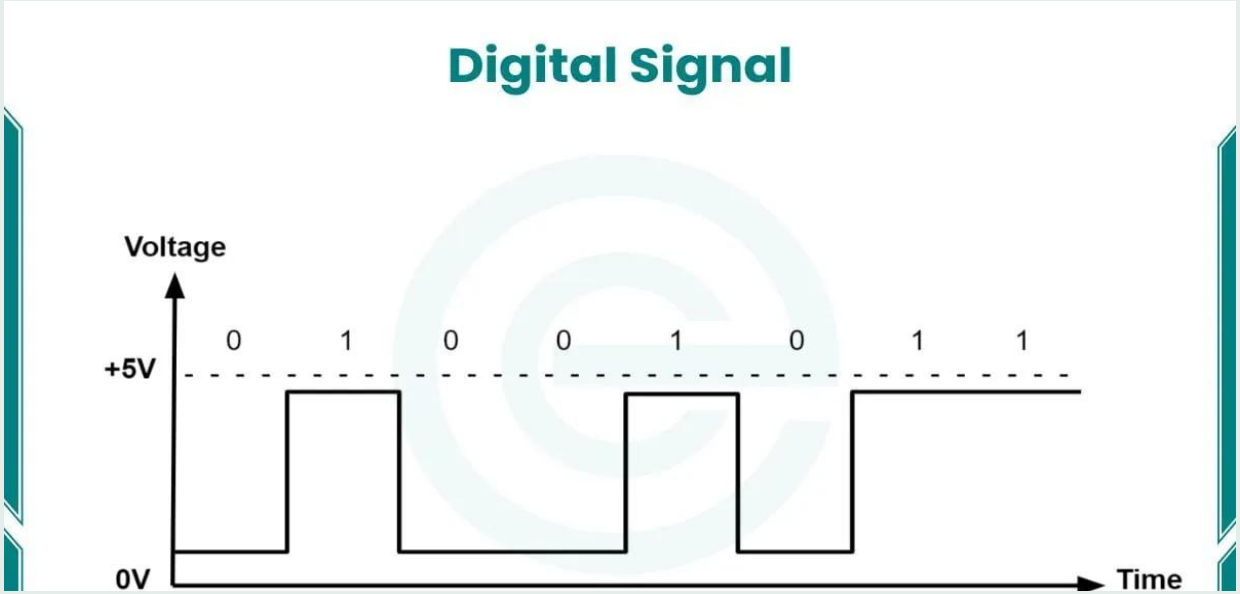
Continuous Time		Discrete Time	
Differential Equations		Difference Equations	
Laplace Transform, Transfer Function	State Space Representation	Z transform, Transfer Function	State Space Representation
Root Locus Analysis	State Space: Modeling, Analysis, Design	Root Locus Analysis	State Space: Modeling, Analysis, Design





Digital Signals

- Digital signals: Digital signals are discrete-time, discrete-amplitude representations of data.



Digital Signals-Data

- Digital Signals data are represented by an array of numbers x[n]
- Index n is integer, why?
- Can Index be negative?

Python data representation

- x=[-1, 0, 1, 2, 2/3, -3/4, 4.5]
- X[2] ?

ADC?

DAC?

Discrete-time signals

- **Discrete-time signals** also have similar representation, however, differ in value of each sample
- Values of **Digital signals**: character, integer, float, or double in Python or C languages
- No such limitation for discrete-time signals
- DSP and control books use the term discrete-time for ease of derivations
- Digital representation is used in practical implementation

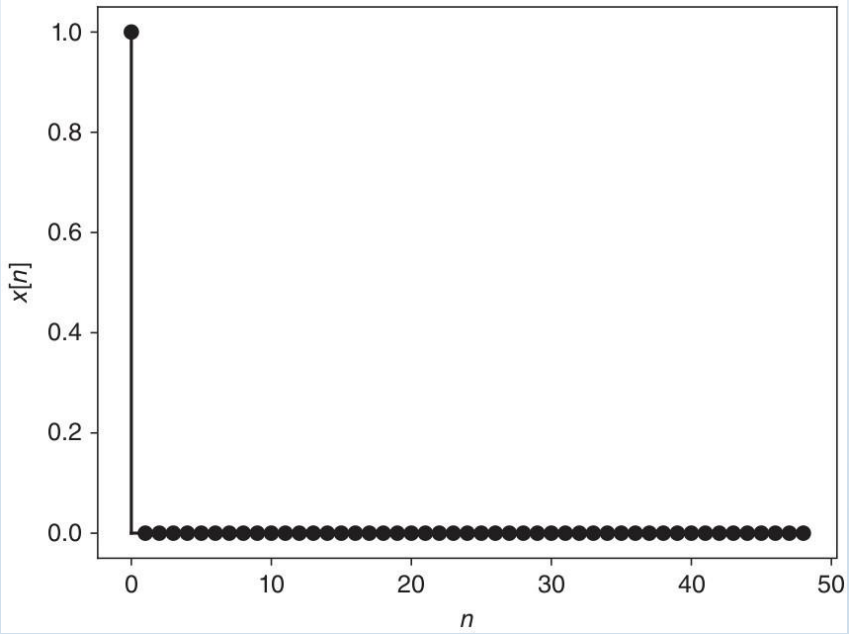
Overview- Fundamentals of Digital Control

- Digital Signals
- Types of Digital Signals
- Digital Systems
- Stability of Digital Systems
- Connections types of Digital Systems

Standard Digital Signals – Unit Pulse Signal

unit_pulse_signal(N)

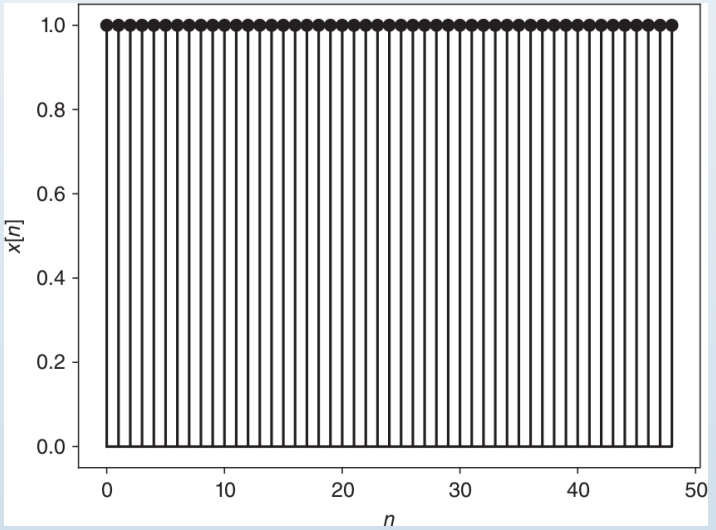
$$\delta[n] = \begin{cases} 1 & n = 0 \\ 0 & \text{otherwise} \end{cases}$$



Standard Digital Signals – Step Signal

- **step_signal(N, Amp)**

$$u[n] = \begin{cases} A & n = 0, 1, 2, \ldots \\ 0 & \text{otherwise} \end{cases}$$

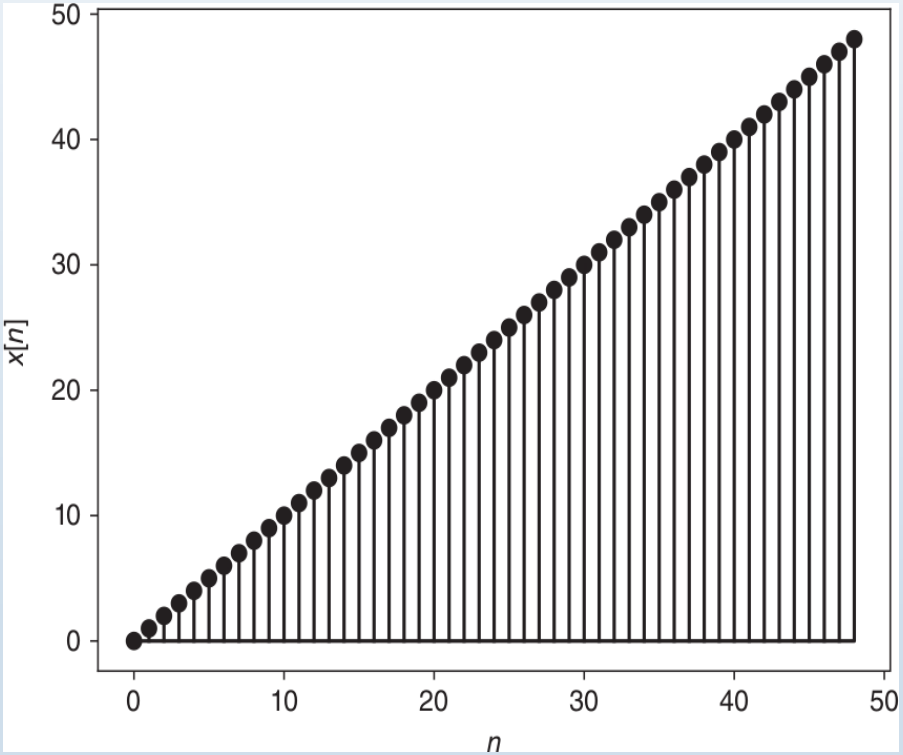


QUIZ?

Standard Digital Signals – Ramp Signal

- **ramp_signal(N, Amp)**

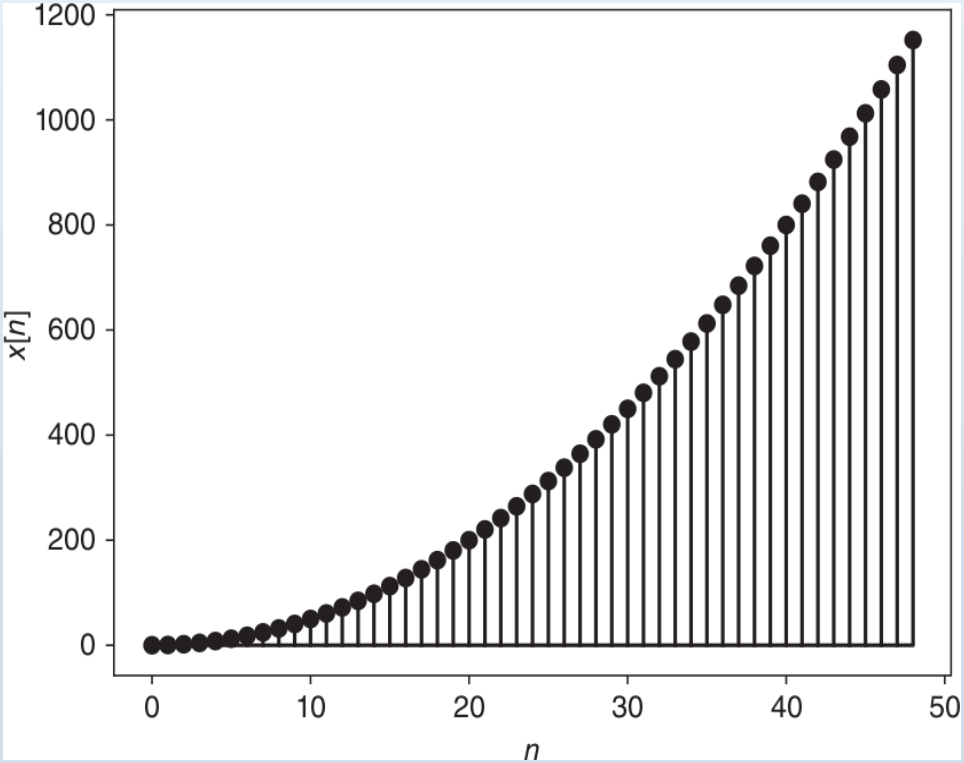
$$r[n] = \begin{cases} An & n = 1, 2, \ldots \\ 0 & \text{otherwise} \end{cases}$$



Standard Digital Signals – Parabolic Signal

- parabolic_signal(N, Amp)

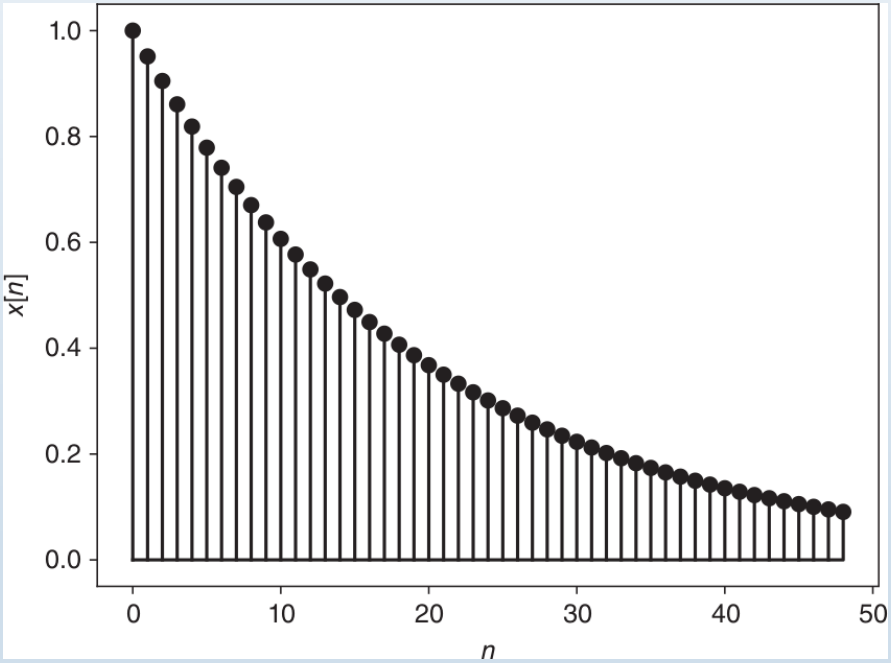
$$r[n] = \begin{cases} An^2 & n = 1, 2, \ldots \\ 0 & \text{otherwise} \end{cases}$$



Exponential Signal

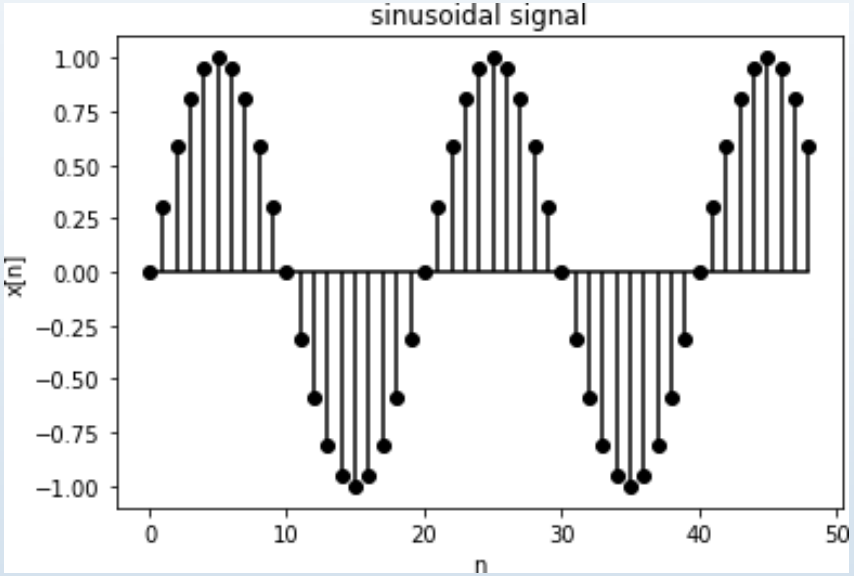
- exponential_signal(N, a)

$$e[n] = \begin{cases} e^{an} & n = 0, 1, 2, \ldots \\ 0 & \text{otherwise} \end{cases}$$



Sinusoidal Signal

- $s[n] = (A \sin(2\pi f / f_s n + \varphi) + C)u[n]$.
  - A is the amplitude,
  - f is the frequency,
  - f_s is the sampling frequency,
  - φ is the phase, and
  - C is the offset term.
- The sinusoidal signal is defined within the MicroPython control systems library as the function




```
sinusoidal_signal(N,Amp, Freq,Phase,Offset,Fsample,select).
```

- The sinusoidal signal is defined within the C control systems library as the function

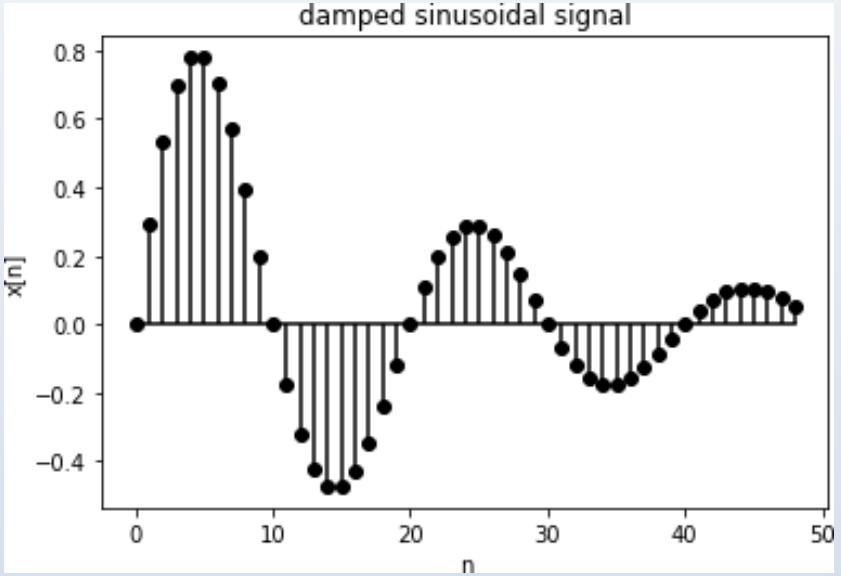
- `void sinusoidal_signal(int N,float Amp,float Freq,float Phase, float Offset,float Fsample, int select,float signal[]).`

Damped Sinusoidal Signal

- $s[n] = e^{an}(A \sin(2\pi f / f_s n + \varphi) + C)u[n]$
- a is the decay parameter, A is the amplitude, f is the frequency, f_s is the sampling frequency, φ is the phase, and C is the offset term.
- The damped sinusoidal signal is defined within the MicroPython control systems library as the function

```
damped_sinusoidal_signal(N,  Amp exp, Amp sin, Freq, Phase, Offset, Fsample, select).
```

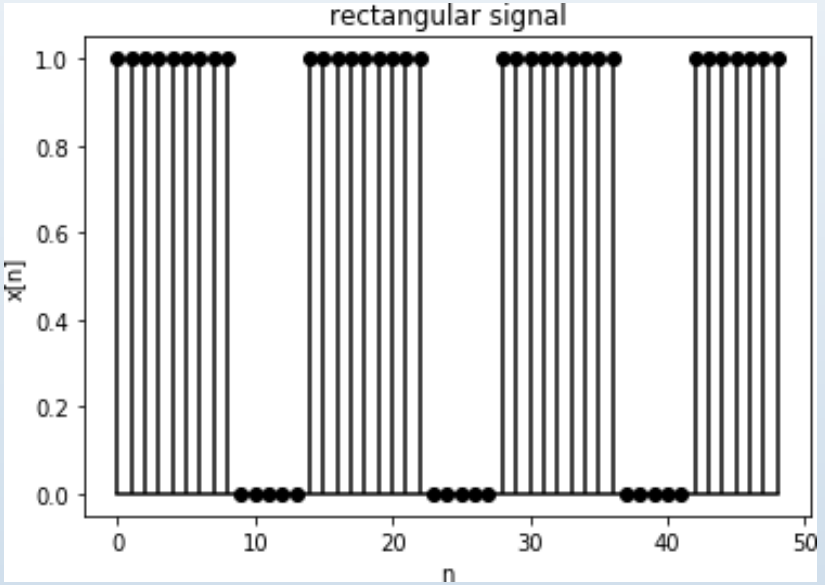
- The damped sinusoidal signal is defined within the C control systems library as the function




```
void damped_sinusoidal_signal(int N,  float Amp exp, float Amp sin, float  Freq, float Phase, float Offset, float  Fsample, int select, float signal[]).
```

Rectangular Signal

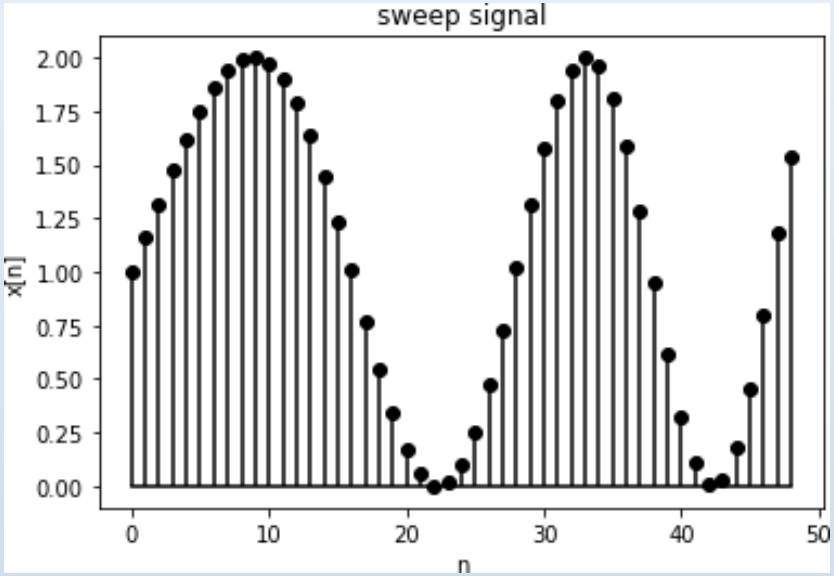
- The rectangular signal is defined within the MicroPython control systems library as the function
- `rectangular_signal(N, Amp, Period, Duty, Offset).`
- The rectangular signal is defined within the C control systems library as the function
- `void rectangular_signal(int N, float Amp, float Period, float Duty, float Offset, float signal[]).`



Sweep Signal

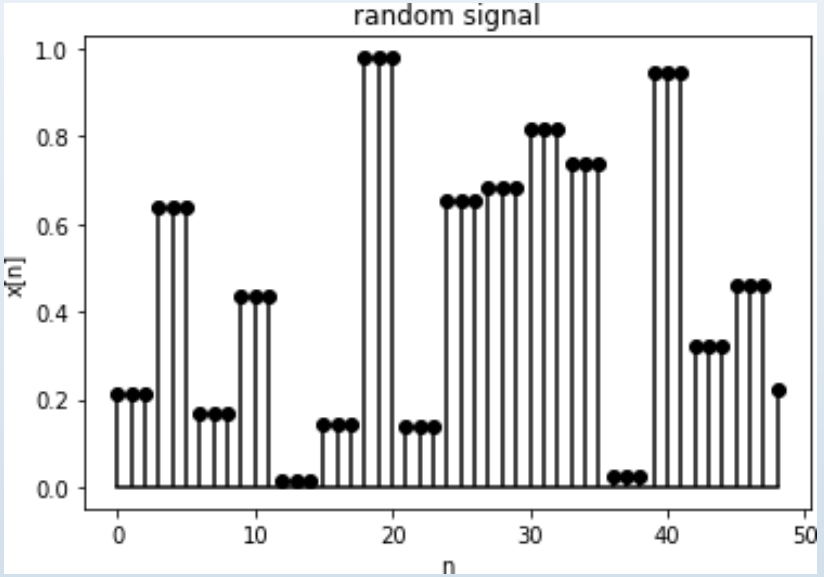
$$(A\sin((2\pi f_1 n/f_s)+(2\pi(f_2-f_1)n^2/(2f_s(d-1))))+C)u[n].$$

- The sweep signal is defined within the MicroPython control systems library as the function
- `sweep_signal(N, Amp, Freq start, Freq stop, Offset, Fsample, Duration).`
- The sweep signal is defined within the C control systems library as the function
- `void sweep_signal(int N, float Amp, float Freq start, float Freq stop, float Offset, float Fsample, float duration, float signal[]).`



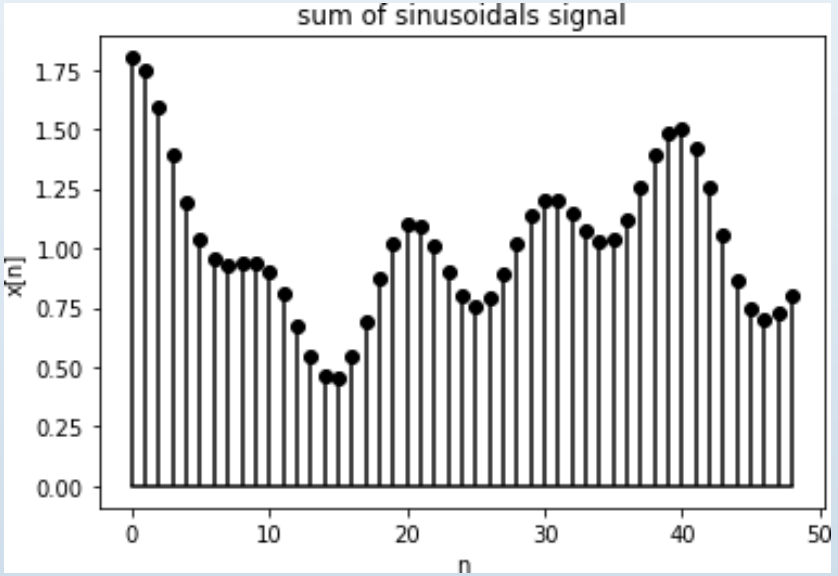
Random Signal

- The random signal is defined within the MicroPython control systems library as the function `random_signal(N, Amp, Duration, Offset)`.
- The random signal is defined within the C control systems library as the function
- `void random_signal(int N, float Amp, int Duration, float Offset, float signal[])`



Standard Digital Signals – Sum of Sinusoids

- A_k is the amplitude, f_k is the frequency, f_s is the sampling frequency, φ_k is the phase, and C_k is the offset term.
- The sum of sinusoids signal is defined within the MicroPython control systems library as the function
- `sum_of_sinusoids_signal(N, No Sines, Amps, Freqs, Phases, Offsets, Fsample, select).`
- The sum of sinusoids signal is defined within the C control systems library as the function `void sum_of_sinusoids_signal(int N, int No Sines, float Amps[], float Freqs[], float Phases[], float Offsets[], float Fsample, int select, float signal[]).`

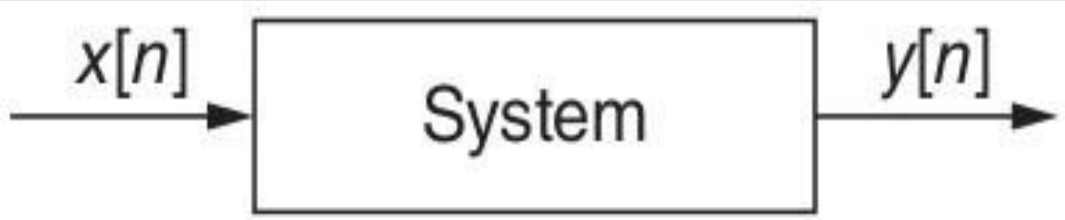


Overview- Fundamentals of Digital Control

- Digital Signals
- Types of Digital Signals
- Digital Systems
- Stability of Digital Systems
- Connections types of Digital Systems

Digital Systems

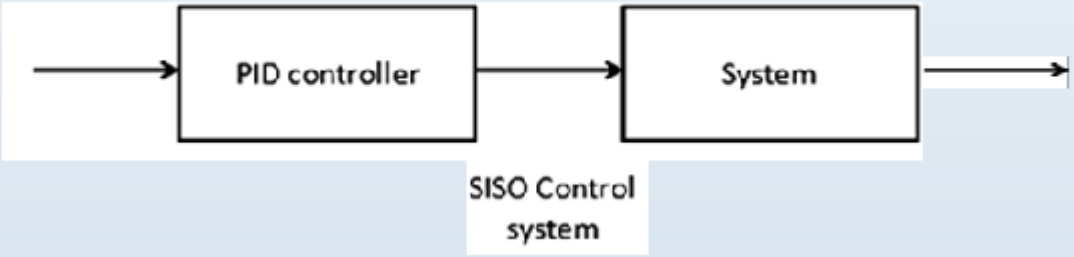
- A digital system is an electronic system that uses discrete values, to process, store, and transmit data and signals. Mathematically, the digital system is an operator producing an output signal to a given input



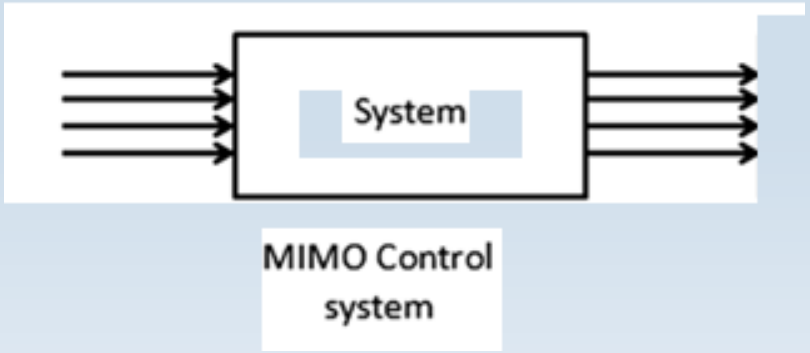
The time is represented and processed not as a smooth, flowing quantity, but as distinct, separate numerical values that jump from one number to the next.

Types: Digital System Types

Single Input and Single Output (SISO)



Multi Input Multi Output (MIMO)



Exp: Drone Flight Controller

- **Inputs:** Sensors (gyroscope, accelerometer, magnetometer, GPS, barometer).
- **Outputs:** Commands to multiple motors (spin speed) and control surfaces.
- **Purpose:** The digital processor (a microcontroller) takes all the sensor readings to determine the drone's orientation and position. It then calculates the precise thrust needed for each motor to keep the drone stable or move it as commanded.

Digital System in Python

- Assume the system
  y[n] = Kx[n],
  where K is a constant

```
def sample_system(x,K):
    y=K*x
    return y
```

Digital System in C

```
void sample_system(float x, float K, float y){
    y=K*x;
}
```

Overview- Fundamentals of Digital Control

- Digital Signals
- Types of Digital Signals
- Digital Systems
- Stability of Digital Systems
- Connections types of Digital Systems

Digital System Properties: Stability

- A system is stable if its output does not diverge to infinity for any given bounded Input
- bounded input means that input does not diverge to infinity
- bounded input bounded output (BIBO)
- Stable system: y[n] = 3x[n]

Specifically, if $|x[n]| \leq M_x$, then $|y[n]| \leq 3M_x$, where M_x is a finite constant.

Stability: Examples

Unstable system: $y[n] = \sum_{k=-\infty}^{n} |x[k]|$

What will be the output of the above systems for

x[n] = u[n] where **u[n]** is the unit step function

as input?

y[n] = 3. u [n]

The output is a scaled step function with 3.

Linear System

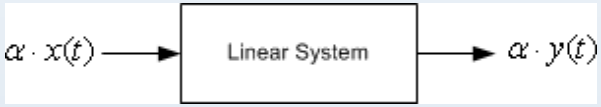
- The relationship between the input and output is linear
- Linearity satisfies the
  - Homogeneity
  - Additivity
  - Superposition conditions

If you were to double the input, the output would also double, assuming everything else remains constant

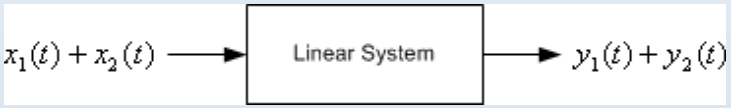
Principle of superposition applies: the response caused by multiple inputs can be found by summing the individual responses to each input

Principal of Superposition

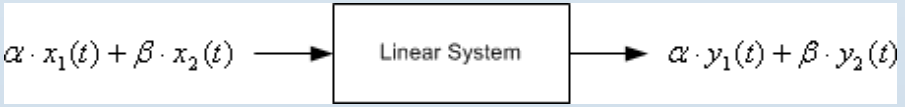
- Homogeneity



- Additivity



- Superposition



Transfer Function

- TF is the relationship between Output & input of the system
- TF is a representation in terms of its input- output
- TF is a convenient way to represent an LTI system

$$H(s) = \frac{Y(s)}{X(s)} = \frac{\mathcal{L}\{y(t)\}}{\mathcal{L}\{x(t)\}}$$

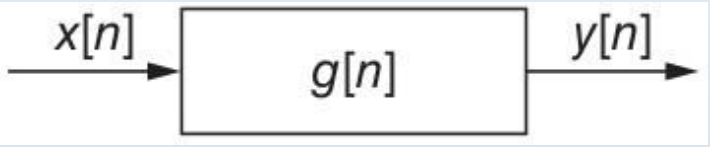
$$H(z) = \frac{Y(z)}{X(z)} = \frac{\mathcal{Z}\{y[n]\}}{\mathcal{Z}\{x[n]\}}$$

z-Transform of H[z] is called Transfer Function of the system

Linear Time-Invariant (LTI) Systems

- input–output relation of an LTI system can be represented by convolution sum

$$y[n] = \sum_{k=-\infty}^{\infty} x[k]g[n-k]$$



Representation of an LTI system with impulse response g[n]

Shorthand representation of the convolution operation is

$y[n] = x[n] * g[n]$.

LTI Systems and Constant-Coefficient Difference Equations

- The input–output relationship of an LTI system can also be represented by its corresponding constant-coefficient general difference equation.

$$y[n] + a_1 y[n-1] + \cdots + a_{L-1} y[n-L+1] = b_0 x[n] + \cdots + b_{K-1} x[n-K+1]$$

- Rewrite The Eq in closed form as

$$y[n] = \sum_{k=0}^{K-1} b_k x[n-k] - \sum_{l=1}^{L-1} a_l y[n-l]$$

- Two sum terms in Eq
  - 1st sum takes current and past input signal values, is called the feedforward term.
  - 2nd sum only takes past output signal values is called the feedback term

LTI System Example: DC Motor

First-order representation of a DC motor:

- $y[n] = 0.9797y[n-1] + 0.2128x[n-1]$
- $H[z] = 0.2128 / (z-0.9797)$

LTI System in Micro Python

```
import mpcontrol
import mpcontrolPC

# System
Ts = 0.0005
num = [0.2128]
den = [1, -0.9797]
g = mpcontrolPC.tf(num, den, Ts)

N = 400
#Step input signal
x = mpcontrol.step_signal(N, 1)

#Calculate the output signal
y = mpcontrol.lsim(g, x)
```

LTI System in Python

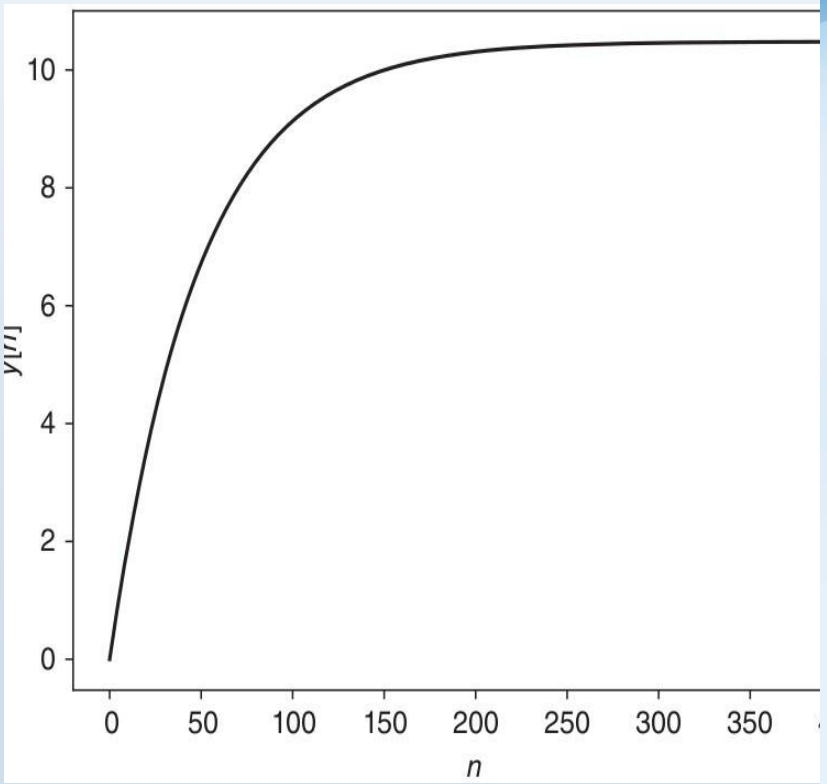

```
import control

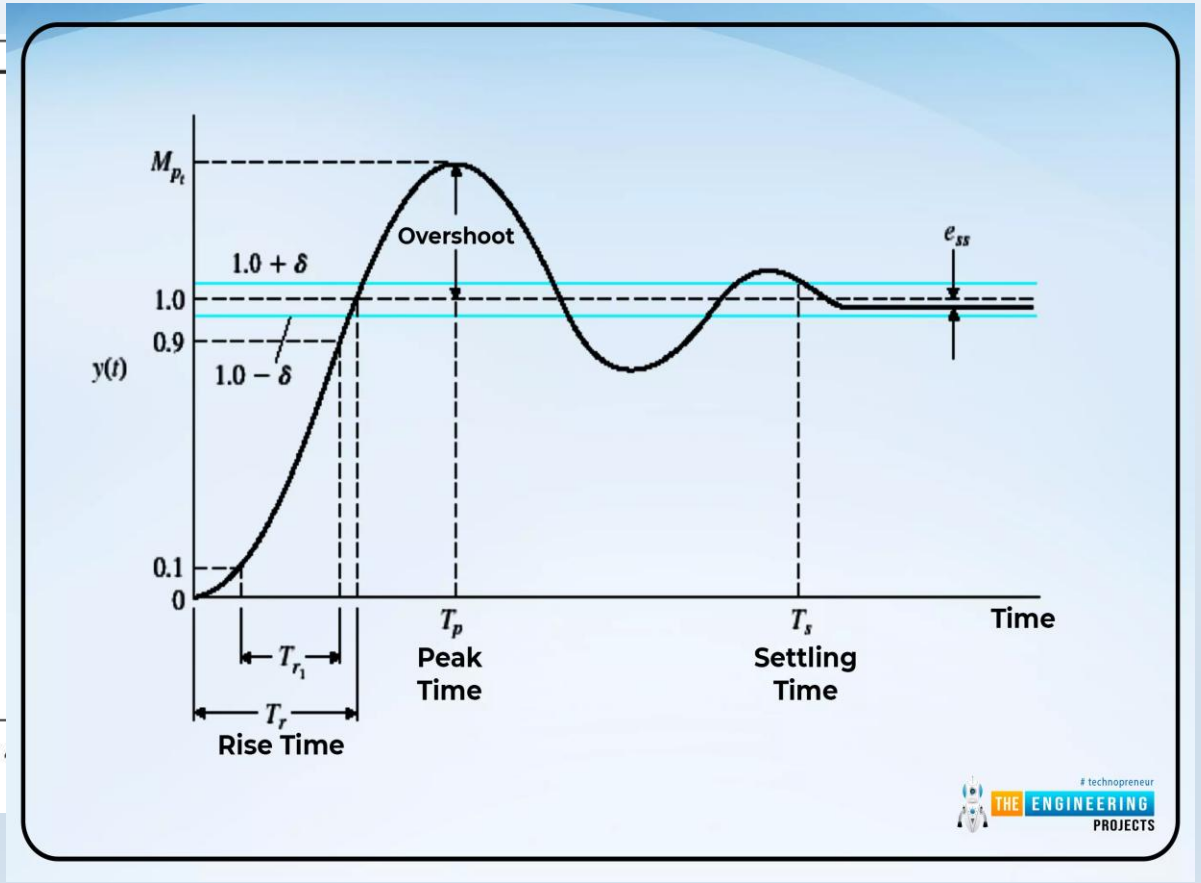
N = 400

# System
Ts=0.0005
num = [0.2128]
den = [1, -0.9797]
t = [n*Ts for n in range(N)]
g = control.TransferFunction(num, den, Ts)

#Calculate step response of the system
n,y=control.step_response(g, T=t)
```

Unit step response of the DC motor





Overview- Fundamentals of Digital Control

- Digital Signals
- Types of Digital Signals
- Digital Systems
- Stability of Digital Systems
- Connections types of Digital Systems

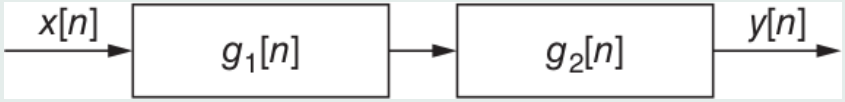
Series Connection

```
import mpcontrolPC

# Systems
Ts = 1
num1 = [1]
den1 = [1, -0.5]
g1 = mpcontrolPC.tf(num1, den1, Ts)

num2 = [1]
den2 = [1, 0.5]
g2 = mpcontrolPC.tf(num2, den2, Ts)

#Form the series connection
g3 = mpcontrolPC.series(g1, g2)
```



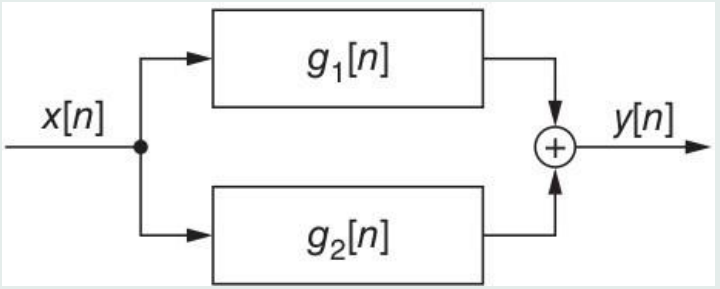
Parallel Connection

```
import mpcontrolPC

# Systems
Ts = 1
num1 = [1]
den1 = [1, -0.5]
g1 = mpcontrolPC.tf(num1, den1, Ts)

num2 = [1]
den2 = [1, 0.5]
g2 = mpcontrolPC.tf(num2, den2, Ts)

#Form the parallel connection
g3 = mpcontrolPC.parallel(g1, g2)
```



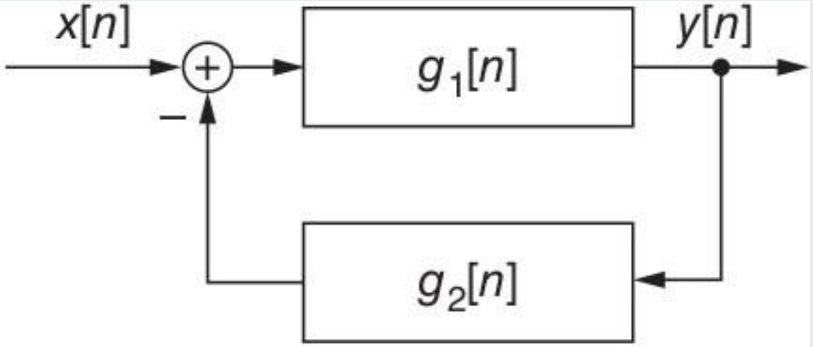
Feedback Connection

```
import mpcontrolPC

# Systems
Ts = 1
num1 = [1]
den1 = [1, -0.5]
g1 = mpcontrolPC.tf(num1, den1, Ts)

num2 = [1]
den2 = [1, 0.5]
g2 = mpcontrolPC.tf(num2, den2, Ts)

#Form the feedback connection
g3 = mpcontrolPC.feedback(g1, g2)
```



Python Demo

- Notebook to demo connections